

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..19

THE BASICS

 Copy as Markdown

Authentication basics

Serverpod automatically checks if the user is signed in and if the user has the right privileges to access each endpoint. When using the authentication module, tokens are tracked, refreshed, and included in requests for you.

The `Session` object provides information about the current user. A `Session` is the request context that every endpoint method receives. It is not the user's login session. Access the current authentication through the synchronous `authenticated` getter of the `Session` object. It exposes a `userIdentifier`, a `String` that uniquely identifies the signed-in user. Use this id whenever you refer to a user.

```
Future<void> myMethod(Session session) async {
  final authenticationInfo = session.authenticated;
  final userIdentifier = authenticationInfo?.userIdentifier;
  ...
}
```

You can also use the `Session` object to check if a user is authenticated:

```
Future<void> myMethod(Session session) async {
  var isSignedIn = session.isUserSignedIn;
  ...
}
```

Requiring authentication on endpoints

To restrict an endpoint to signed-in users, override the `requireLogin` property of the `Endpoint` class.

```
class MyEndpoint extends Endpoint {
  @override
  bool get requireLogin => true;
}
```



Ask AI

```
Future<void> myMethod(Session session) async {
  ...
}
...
}
```

Explicitly allowing unauthenticated access

In some cases, you may want to explicitly allow certain endpoints or methods to be accessed without authentication. Serverpod provides the `@unauthenticatedClientCall` annotation for this purpose.

When an endpoint or method is annotated with `@unauthenticatedClientCall`:

- No authentication will be added to the header on the client when calling it.
- The server will treat the call as unauthenticated, regardless of any existing authentication state.

You can use this annotation in two ways:

1. On the entire endpoint class to make all methods unauthenticated:

```
@unauthenticatedClientCall
class UnauthenticatedEndpoint extends Endpoint {
  Future<bool> someMethod(Session session) async {
    return session.isUserSignedIn; // Will always return false
  }

  Stream<bool> someStream(Session session) async* {
    yield session.isUserSignedIn; // Will always return false
  }
}
```

2. On specific methods to make only those methods unauthenticated:

```
class PartiallyUnauthenticatedEndpoint extends Endpoint {
  @unauthenticatedClientCall
  Future<bool> publicMethod(Session session) async {
    return session.isUserSignedIn; // Will always return false
  }

  Future<bool> authenticatedMethod(Session session) async {
    return session.isUserSignedIn;
  }
}
```

This is particularly useful for endpoints that must not receive authentication, such as the [JWT refresh endpoint](#).

⚠ WARNING

Using `@unauthenticatedClientCall` on an endpoint or method that also has `requireLogin` set to true will lead to a conflict. The client suppresses the authentication header, but the server expects it, so calls to such endpoints or methods always fail with an authentication error.

Authorization on endpoints

Scopes define what an authenticated user is allowed to do. Each scope is an independent capability identified by a string name. Scopes do not inherit from each other. Holding `Scope.admin` does not grant any other scope.

Each endpoint exposes two properties for access control:

- `requireLogin`: the user must be signed in, with no specific scope required.
- `requiredScopes`: the user must hold every scope in the set.

To restrict access by scope, override the `requiredScopes` property:

```
class MyEndpoint extends Endpoint {  
  @override  
  Set<Scope> get requiredScopes => {Scope.admin};  
  
  Future<void> myMethod(Session session) async {  
    ...  
  }  
  ...  
}
```

Serverpod ships with `Scope.admin` for built-in admin functionality in Serverpod modules. Define your own scope names for application-specific access control.

When `requiredScopes` is non-empty, authentication is required even if `requireLogin` is false.

Custom scopes

Define constants for your domain by extending the `Scope` class:

```
class CustomScope extends Scope {  
  const CustomScope(String name) : super(name);  
  
  static const userRead = CustomScope('userRead');  
  static const userWrite = CustomScope('userWrite');  
}
```

Then use the custom scopes on your endpoints:

```
class MyEndpoint extends Endpoint {
  @override
  Set<Scope> get requiredScopes => {CustomScope.userRead, CustomScope.userWrite};

  Future<void> myMethod(Session session) async {
    ...
  }
  ...
}
```

The user must hold both `userRead` and `userWrite` to access this endpoint. You can reuse the same scope constants across multiple endpoints to build different access combinations.

Keep scope names stable once deployed, as renaming a scope will revoke it from any user who had it.

You can also define shared scope requirements in a base endpoint class. See [Endpoint inheritance](#) for details.

CAUTION

A scope is identified by its string name only. If two scope constants share the same string, granting one also grants the other, since the server cannot tell them apart. Give every scope a unique string.

How scopes combine

When an endpoint lists multiple scopes in `requiredScopes`, the user must have all of them. Serverpod evaluates scopes with AND logic. There is no OR matching and no scope hierarchy.

Consider a user management feature with two admin endpoints: one for aggregated user analytics and one for editing user data. Not every admin should be able to modify users, so the endpoints require different scope combinations:

```
class UserAnalyticsEndpoint extends Endpoint {
  @override
  Set<Scope> get requiredScopes => {Scope.admin};

  Future<UserStats> getStats(Session session) async {
    ...
  }
}

class UserEditEndpoint extends Endpoint {
  @override
  Set<Scope> get requiredScopes => {
    Scope.admin,
    CustomScope.userWrite,
  };
}
```

```
Future<void> updateUser(Session session, UserData data) async {
  ...
}
```

An admin user with only `Scope.admin` can call `UserAnalyticsEndpoint` but not `UserEditEndpoint`. To allow editing, grant the user both scopes, as shown in [Managing scopes](#) below. This lets you compose capabilities at the endpoint level instead of building nested roles.

Managing scopes

New users are created without any scopes.

To update a user's scopes, use the `update` method from `AuthServices.instance.authUsers`. This method replaces all previously stored scopes:

```
import 'package:serverpod_auth_idp_server/core.dart';

await AuthServices.instance.authUsers.update(
  session,
  authUserId: authUserId,
  scopes: {Scope.admin},
);
```

Changing a user's scopes does not affect existing sessions or tokens until the user signs in again or their tokens are revoked. Serverpod closes open method streams when a revoked scope overlaps what the endpoint requires. See [Managing tokens](#) for revoking tokens across devices.

HTTP responses

When access is denied, Serverpod returns:

- **401 Unauthorized:** no token was provided, or the token is invalid.
- **403 Forbidden:** the user is authenticated but missing one or more required scopes.

Client-side authentication

On the client side, authentication state is managed through the `FlutterAuthSessionManager`, which is accessible via `client.auth`.

NOTE

On macOS, the `FlutterAuthSessionManager` stores tokens in the Keychain. New projects need a Keychain Sharing entitlement before sign-in works. See [Set up authentication on macOS](#).

NOTE

If you are building a pure Dart application, use the `ClientAuthSessionManager` from the `serverpod_auth_core_client` package instead. It works the same way, except it has no `authInfoListenable` getter, which is tied to the Flutter framework.

Check authentication state

To check if the user is signed in:

```
bool isSignedIn = client.auth.isAuthenticated;
```

Returns `true` if the user is signed in, or `false` otherwise.

Access current authentication info

To retrieve information about the current authentication:

```
AuthSuccess? authInfo = client.auth.authInfo;
```

Returns an `AuthSuccess` object if the user is currently signed in, or `null` if the user is not.

Register authentication

To register a signed-in user, call:

```
await client.auth.updateSignedInUser(authInfo);
```

This persists the authentication information. Identity providers call this method to register a signed-in user. For more details on providers, see [Custom Providers](#).

When the signed-in user changes, open method streams close, and new streams connect as the current user. A token refresh for the same user keeps them open.

Monitor authentication changes

The `FlutterAuthSessionManager` exposes `authInfoListenable`, a `ValueListenable<AuthSuccess?>`. Listen to it to update the UI whenever the user signs in or out:

```
@override
void initState() {
  super.initState();

  // Listen to authentication state changes.
```

```

    client.auth.authInfoListenable.addListener(_onAuthStateChanged);
}

// Don't forget to remove the listener when the widget is disposed.
@override
void dispose() {
    client.auth.authInfoListenable.removeListener(_onAuthStateChanged);
    super.dispose();
}

void _onAuthStateChanged() {
    setState(() {
        // UI will rebuild when auth state changes.
    });
}

```

The listener is triggered whenever the user's sign-in state changes.

Validate the session and handle expiry

Serverpod refreshes tokens automatically while the user stays signed in, so most apps never handle tokens directly. Expiry becomes visible only when the refresh token itself has expired or been revoked: the client can no longer refresh, and the stored session is no longer valid.

Call `validateAuthentication` to check the current session against the server and sign the user out if it is no longer valid:

```

await client.auth.validateAuthentication(); // throws on transient errors; retry if
needed

```

The method force-refreshes the token and confirms with the server that the user is still signed in. If the session is no longer valid, it signs the user out on the current device. A transient problem, such as a network error or timeout, does not sign the user out. The exception is thrown instead, so you can catch it and retry.

At app startup, use `initialize` to restore a stored session and validate it in one step:

```

await client.auth.initialize();

```

The `initialize` method runs `restore` followed by `validateAuthentication`. It returns `true` when validation completed, which is not the same as the user being signed in. Read `client.auth.isAuthenticated` for that. If the stored session has expired, the user is signed out. If validation cannot complete because of a network or server error, `initialize` returns `false` and leaves the stored session in place so you can retry later, which keeps offline users signed in. If the validation call times out, the timeout is thrown instead, so catch it if you want the same retry behavior.

Because signing out updates the authentication state, a listener registered on `authInfoListenable` (see [Monitor authentication changes](#)) fires when a session expires, so you can route the user back to a sign-in screen from one place.

Signing out users

The `FlutterAuthSessionManager` provides methods for handling user sign-outs, whether from a single device or all devices. They call the `StatusEndpoint` methods under the hood, which are also directly accessible on the client through the `client.modules.serverpod_auth_core.status` getter. For revoking authentication across devices from the server, see [Managing tokens](#).

Sign out current device

To sign the user out from the current device:

```
await client.auth.signOutDevice();
```

Returns `true` if the server call succeeds, or `false` if it fails. Either way, the app clears its local authentication state.

Sign out all devices

To sign the user out across all devices:

```
await client.auth.signOutAllDevices();
```

Returns `true` if the user is successfully signed out from all devices, or `false` if it fails. Either way, the app clears its local authentication state.

Related

- [Setup](#): install and configure the authentication module.
- [Working with users](#): user profiles, callbacks, and admin operations.
- [Token managers](#): how tokens are issued, validated, and revoked.
- [Custom overrides](#): replace the built-in authentication with your own.